ISABELLA'S · MENÚ 2026 · INFORME PARA PROPUESTA COMERCIAL

Cómo administrar la carta sin contratar funciones de pago

Comparación de ocho caminos para que el restaurante edite platos, precios, imágenes y cinco idiomas. Recomendación para la web actual y alternativa si no se puede ampliar WordPress.

Investigado el 16 de octubre de 2026 · Documento editable · Vista previa local, sin cambios en producción

Decisión recomendada

> **Usar el WordPress que ya tiene Isabella y desarrollar un plugin propio de menú.** El cliente editaría desde un apartado «Carta» con permisos limitados; el frontend de revista leería datos publicados por WordPress. No requiere licencia de CMS, plan Growth ni un segundo servidor. Sí requiere desarrollo, pruebas, copias de seguridad y el alojamiento WordPress que ya paga el sitio. WordPress ofrece tipos de contenido, campos, medios, borradores, revisiones, roles y API para este modelo. [3–6]

Condición antes de comprometer la implementación: confirmar con el responsable del sitio que se pueden instalar plugins propios en un entorno de pruebas, que se dispone de acceso técnico y de un respaldo restaurable. No se comprobó el panel privado, el contrato de hosting ni permisos de instalación. Si esa condición falla, la segunda opción de menor fricción es un proyecto **Sanity Free dedicado exclusivamente al menú**, aceptando que quien edite tendrá rol de administrador del proyecto. [7]

Lo pedido en la reunión y lo comprobado

En la grabación de Jam, alrededor de 1:37–2:48, el jefe plantea que el cliente agregue platos y modifique precios, nombres, descripciones e imágenes. Pide comparar Sanity con crear otra aplicación administrativa sobre Firebase y preparar un análisis para cotizar. Antes aclara que la primera versión es un menú de consulta, con español, inglés, alemán, árabe y hebreo, selección de idioma y lectura RTL. La transcripción de Jam es automática; se usa su sentido claro y se evitan términos ambiguos. [1]

La web pública de Isabella's muestra recursos de WordPress/Astra, el enlace `/wp-json/` y el visor dFlip. Esta constatación pública justifica evaluar primero la infraestructura existente; no demuestra acceso al administrador ni capacidad de instalar plugins. El preview local contiene 55

platos en ocho categorías derivados de las nueve páginas «MENU 2026»; cinco importes siguen sin verificación. [2]

Opciones gratuitas examinadas

«Gratis» aquí significa sin licencia o función de pago obligatoria para editar esta carta. No significa que desarrollo, hosting, soporte o traducción humana sean gratuitos. Los límites de planes pueden cambiar y se deben reconfirmar antes de contratar o desplegar.

Opción	Lo que resuelve sin pagar licencia	Límite o trabajo real	Encaje para Isabella
WordPress propio	Tipos de plato/categoría, biblioteca de medios, roles, borradores, revisiones y REST; aprovecha la web existente. [3–6]	Hay que construir el plugin, el formulario multidioma, validaciones y conexión del preview. Requiere acceso técnico y pruebas.	**Recomendado** Menor dependencia nueva y permisos editoriales acotables.
Sanity Free	Studio y base alojados; hasta 20 asientos, dos datasets públicos, locales y vista previa. [7]	Free solo ofrece Administrator/Viewer; Editor no está incluido. El cliente tendría administración completa de un proyecto dedicado, y aún hay que integrar el frontend.	Buena alternativa rápida si WordPress no admite el plugin; separar el proyecto limita el alcance del rol.
Strapi Community	CMS MIT con internacionalización, roles, medios y borrador/publicación en el plan gratuito. [8]	Debe alojarse, actualizarse y respaldarse aparte; requiere Node y base de datos.	Mejor opción autohospedada independiente si ya existe servidor compatible.
Directus Core	Plan de software de $0 con tres asientos y 25 colecciones; su interfaz admite traducciones. [9]	Alojamiento aparte. Más asientos o ciertas necesidades superan Core; existe una beca gratuita condicionada a elegibilidad.	Viable, pero añade operación y condiciones de plan sin aportar ventaja decisiva.
Payload	Código abierto con panel, localización por campo, acceso y archivos. [10]	Requiere aplicación Node/Next, base de datos, almacenamiento persistente y mantenimiento.	Potente si se construye una plataforma mayor; excesivo para una carta de consulta.
Decap CMS	Editor abierto sobre Git con medios y flujo de borrador/revisión. [11]	Necesita repositorio, autenticación OAuth y despliegue al publicar; se acopla peor al WordPress actual.	Útil para sitio estático nuevo, menos cómodo para este cliente.

Opción	Lo que resuelve sin pagar licencia	Límite o trabajo real	Encaje para Isabella
Supabase Free + panel propio	Postgres, autenticación y almacenamiento dentro de cuota gratuita. [12]	No trae el editor gastronómico: hay que construirlo. Free carece de backups automáticos y pausa tras una semana sin actividad.	No elegir para producción del menú con esta restricción de coste.
Firebase Spark + panel propio	Firestore tiene cuota gratuita; autenticación y reglas pueden sustentar una app propia. [13]	Hay que construir todo el administrador. Cloud Storage para imágenes exige activar Blaze y cuenta de facturación, aunque haya uso sin cargo.	No cumple bien la petición de evitar funciones con facturación.

Inferencia de esta comparación: para 55 platos, cinco idiomas y una web ya montada en WordPress, el coste y riesgo extra de operar otra plataforma pesan más que el ahorro de construir algunos formularios. Si el restaurante pide más adelante pedidos, mesas o reglas complejas, conviene reabrir la arquitectura; esa funcionalidad no forma parte de la primera versión de Jam.

Diseño del administrador recomendado

Datos compartidos

Un registro por plato: identificador estable, categoría, orden, visibilidad, precio entero en colones, variantes y adicionales con su propio precio, imagen principal y referencia a la página fuente. Ocho categorías editables, con su imagen artística independiente. El precio se cambia una vez y sirve a todos los idiomas.

Campos por idioma

Nombre, descripción y texto alternativo de imagen para ES, EN, DE, AR y HE. Pestañas de idioma con contador de campos pendientes; entradas árabes y hebreas con dirección RTL. No publicar una traducción vacía ni presentar traducción automática como validada.

Permisos y publicación

Rol «Editor de carta» con capacidad sobre platos, categorías y medios del menú, sin acceso a plugins ni ajustes del sitio. Borrador, vista previa y publicación; revisión de cambios y respaldo antes de migrar. El endpoint público solo entrega platos visibles y traducciones aprobadas. [3–6]

Integración con la revista

El frontend existente conserva diseño, vuelta de hoja y RTL, pero lee un JSON público del plugin. Las ocho ilustraciones de categoría pueden quedar como recurso del diseño y cambiarse desde la biblioteca si el cliente lo requiere. Mantener el libro dFlip existente hasta aprobar el nuevo menú.

Implementar el tipo de contenido en un **plugin propio**, no en el tema, sigue la recomendación de WordPress y evita perder el modelo al cambiar Astra. Los campos necesarios se registran para REST y revisiones; las capacidades se asignan al rol del menú. [3–6]

Prueba de aceptación antes de publicar

1. El cliente cambia un precio de ejemplo en borrador, lo revisa y lo publica; aparece igual en los cinco idiomas y no altera los demás platos.
2. Agrega un plato con foto y variantes, lo ordena en su categoría y lo oculta; los cambios de borrador no se ven públicamente.
3. Edita nombre y descripción en árabe y hebreo; el formulario y la carta muestran RTL sin invertir el importe en colones.
4. Recupera una versión anterior de un plato y confirma que la imagen, el precio y los textos esperados vuelven correctamente.
5. Comprueba los 55 platos y resuelve los cinco precios pendientes con el restaurante, sin inventar importes.

Estado hoy: investigación y arquitectura documentadas; no hay CMS conectado ni cambios en el administrador real. Para cotizar ejecución faltan acceso a staging/hosting, confirmación de quién tendrá rol editorial, aprobación de traducciones, fotografías y precios pendientes.

Fuentes primarias consultadas

[1] Grabación y transcripción de Jam, 1:37–2:48.

[2] Web oficial de Isabella's, inspección pública.

[3] WordPress: tipos de contenido en plugins. [4] Campos, revisiones y capacidades. [5] Roles y capacidades. [6] API REST.

[7] Sanity: planes, roles, cuotas y datasets.

[8] Strapi: plan Community; biblioteca de medios.

[9] Directus: Core y beca; traducciones.

[10] Payload: localización; requisitos de despliegue.

[11] Decap: CMS sobre Git; flujo editorial.

[12] Supabase: límites Free, pausas y backups.

[13] Firestore: cuota gratuita; Storage: requisito Blaze.